

INTERVIEW PREPARATION GUIDE

Next.js Interview Questions & Answers

A complete, structured reference covering the App Router, rendering strategies, Server & Client Components, data fetching and caching, Server Actions, middleware, SEO, performance, security, deployment, and hands-on coding challenges — for Junior to Senior level interviews.

125

QUESTIONS

18

TOPIC AREAS

60+

CODE EXAMPLES

Compiled reference · August 2026

Table of Contents

1. Fundamentals & Architecture	Q1–8	2. Pages Router vs App Router	Q9–14
3. File-based Routing (App Router)	Q15–23	4. Rendering Strategies	Q24–32
5. Server & Client Components	Q33–40	6. Data Fetching & Caching	Q41–49
7. Server Actions & Mutations	Q50–55	8. Route Handlers (API Routes)	Q56–61
9. Middleware	Q62–66	10. Layouts, Loading & Streaming	Q67–73
11. Image, Font & Script Optimization	Q74–79	12. SEO & Metadata	Q80–85
13. Styling Solutions	Q86–90	14. Performance Optimization	Q91–97
15. Authentication & Security	Q98–104	16. Deployment & Runtimes	Q105–110
17. Testing & Debugging	Q111–115	18. Practical Coding Challenges	Q116–125

1 Fundamentals & Architecture

What Next.js is, and why it exists on top of React.

Q1. What is Next.js, and why use it instead of plain React? BEGINNER

Next.js is a React framework (by Vercel) that adds the production infrastructure React itself doesn't provide: file-based routing, multiple rendering strategies (SSR/SSG/ISR), Server Components, built-in image/font optimization, API/Route Handlers, middleware, and a zero-config build pipeline. Plain React (via Vite/CRA) is just a UI library — you'd have to hand-assemble a router, a data-fetching/caching strategy, a bundler config, and an SSR setup yourself. Next.js bundles opinionated, optimized defaults for all of that.

Q2.

Next.js vs Create React App / Vite + React — what's actually different?

BEGINNER

- CRA/Vite ship a **client-only** single-page app by default — the browser downloads JS, then renders. Next.js renders on the server (or at build time) first, so users see content before JS loads.
- Next.js has **file-based routing** built in; a Vite SPA needs a separate router library (react-router) configured manually.
- Next.js supports mixed rendering per-route (static, server-rendered, or incremental) in one app; a Vite SPA is client-rendered everywhere unless you add your own SSR layer.
- Next.js includes image optimization, font optimization, and route-level code-splitting out of the box.

Q3. What are the main features Next.js provides on top of React?

BEGINNER

- **File-based routing** — folders and files under `app/` (or `pages/`) become routes automatically.
- **Multiple rendering modes** — SSR, SSG, ISR, and CSR, chosen per route.
- **React Server Components** — render components on the server with zero client JS by default.
- **Built-in optimization** — `next/image`, `next/font`, `next/script`, automatic code-splitting.
- **API/Route Handlers** — backend endpoints colocated with the frontend code.
- **Middleware** — logic that runs before a request completes (auth, redirects, A/B tests).
- **Zero-config tooling** — bundling, TypeScript support, and fast refresh with no manual webpack setup.

Q4. What is hydration, and why does Next.js need it? INTERMEDIATE

The server sends fully-rendered HTML so the page is visible immediately, but that HTML is inert — no event listeners are attached yet. **Hydration** is the process where React runs in the browser, reuses the existing server-rendered DOM, and attaches the event handlers/state needed to make the page interactive, without re-rendering everything from scratch. If the HTML React expects doesn't match what the server actually sent (a **hydration mismatch**), React logs a warning and re-renders the mismatched portion on the client.

Q5. What actually happens when you run next build? INTERMEDIATE

- Next.js compiles and type-checks the app, bundling client and server code separately.
- It analyzes every route to decide whether it can be **statically prerendered** (no dynamic data/request-time APIs used) or must be **server-rendered on demand**.
- Static routes are rendered to HTML at build time and written to disk; dynamic routes are compiled into serverless/edge functions.
- It produces an optimized production bundle (minified, tree-shaken, code-split per route) and prints a route-by-route summary (Static ○, SSG ●, SSR λ, ISR) so you can see how each page will behave in production.

Q6.**What is Turbopack, and how does it differ from Webpack in Next.js?**INTERMEDIATE

Turbopack is a Rust-based bundler (built by the Next.js/Vercel team) designed as a much faster successor to Webpack, used for `next dev` (and increasingly `next build`). It uses function-level incremental caching, so on a file change it only recompiles the parts of the dependency graph actually affected, rather than re-bundling broadly — giving significantly faster cold starts and hot-reload times than Webpack on large apps, while aiming for drop-in config compatibility.

Q7.**How does Next.js compare to Remix or other React meta-frameworks?****ADVANCED**

Both are full-stack React frameworks with file-based routing and server rendering. Key differences: Next.js's App Router is built around **React Server Components** as the default, with granular per-segment caching (Data Cache, Full Route Cache); Remix leans more on standard **Web Fetch APIs** and nested-route data loading (loaders/actions) without RSC as a first-class concept (though it's adding support). Next.js has tighter, first-party integration with Vercel's edge infrastructure; Remix is more deployment-target-agnostic by design. In practice the choice often comes down to team familiarity, hosting target, and how much you want the RSC caching model versus a more classic loader/action data flow.

Q8. What are the main Next.js CLI commands?**BEGINNER****BASH**

```
next dev      # start the local dev server with hot reload
next build    # create an optimized production build
next start    # run the production build (needs `next build` first)
next lint     # run ESLint with Next.js's recommended rules
```

2 Pages Router vs App Router

Q9.

What is the Pages Router, and how does its file-based routing work?

BEGINNER

The original Next.js routing system: every file under `pages/` automatically becomes a route based on its file path — `pages/about.js` → `/about`, `pages/blog/[slug].js` → `/blog/:slug`. Data fetching is done via special exported functions (`getStaticProps`, `getServerSideProps`), and `_app.js` / `_document.js` control the shared app shell and HTML document. It's still fully supported, but the App Router is now the recommended default for new projects.

Q10.

What is the App Router, and what does it add over the Pages Router?

INTERMEDIATE

Introduced in Next.js 13 and now the default (`app/` directory), the App Router is built on **React Server Components** and adds: nested layouts that persist across navigations, colocated `loading.tsx` / `error.tsx` boundaries per route segment, streaming with Suspense, Server Actions for mutations without a separate API layer, and a more granular, fetch-based caching model — all things the Pages Router either couldn't do or needed extra libraries for.

Q11.

Can you use both the Pages Router and the App Router in the same project?

INTERMEDIATE

Yes — Next.js supports incremental adoption. Both `app/` and `pages/` can coexist; routes are resolved by matching the URL against both directories (App Router takes precedence on conflicts). This is the officially recommended migration path: move routes to `app/` one at a time rather than doing a big-bang rewrite.

Q12.

What are the key file-convention differences between `pages/` and `app/`?

INTERMEDIATE

- `pages/_app.js` (shared shell) → `app/layout.tsx` (root layout, and any nested `layout.tsx`).
- `pages/_document.js` (custom HTML doc) → handled inside `app/layout.tsx`'s `<html>` / `<body>`.
- `getStaticProps` / `getServerSideProps` → plain `async` Server Components using `fetch()` with caching options.
- `pages/api/*.js` → `app/api/*/route.ts` Route Handlers.
- `pages/404.js` → `app/not-found.tsx`.
- Every component in `app/` is a Server Component by default, unlike `pages/` where everything renders (and hydrates) as a Client Component.

Q13.

Why did Next.js introduce the App Router — what problem was it solving?

ADVANCED

The Pages Router sends every component's JS to the client, even parts that never need interactivity, and its per-page data-fetching functions made nested-layout data fetching and partial page updates awkward (a shared layout re-fetches/re-renders unnecessarily on every navigation). The App Router was built around **React Server Components** specifically to let the framework render as much as possible on the server (shipping zero JS for non-interactive parts), support layouts that persist and don't re-fetch on child navigation, and enable streaming so slow parts of a page don't block the fast parts from showing up.

Q14.**Is the Pages Router deprecated? Should a new project use the App Router?****BEGINNER**

The Pages Router is not deprecated — it's stable and still maintained — but the App Router is the actively-developed, recommended path for new projects, since new features (Server Actions, Partial Prerendering, the Metadata API) are built for it first. For a brand-new app, start with the App Router; for an existing large Pages Router app, incremental migration rather than a rewrite is the recommended approach.

3 File-based Routing (App Router)

Q15. How does file-based routing work in the App Router?

BEGINNER

Every **folder** under `app/` defines a URL segment, and a `page.tsx` file inside that folder makes the segment publicly routable (folders without a `page.tsx` are not directly navigable, which lets you colocate components/utilities without accidentally creating routes).

PLAINTEXT

```
app/  
  page.tsx      -> /  
  about/  
    page.tsx    -> /about  
  blog/  
    page.tsx    -> /blog  
    [slug]/  
      page.tsx  -> /blog/:slug
```

Q16.**How do dynamic routes work, and how do you read the route parameter?****BEGINNER**

A folder name wrapped in square brackets, e.g. `[id]`, creates a dynamic segment. The value is passed to the page as part of the `params` prop.

TSX

```
// app/products/[id]/page.tsx
export default async function ProductPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const product = await getProduct(id);
  return <h1>{product.name}</h1>;
}
```

Q17.**What's the difference between catch-all and optional catch-all routes?****INTERMEDIATE**

`[...slug]` (catch-all) matches one or more path segments, e.g. `/docs/a/b/c` → `slug = ['a', 'b', 'c']`, but does **not** match `/docs` itself. `[...slug]` (optional catch-all) also matches zero segments, so `/docs` matches too, with `slug = undefined`. This is commonly used for CMS-driven pages where the path depth is unknown ahead of time.

PLAINTEXT

```
app/docs/[...slug]/page.tsx    // matches /docs/a, /docs/a/b (not /docs)
app/docs/[...slug]/page.tsx    // also matches /docs itself
```

Q18. What are route groups, and what problem do they solve?

INTERMEDIATE

A folder wrapped in parentheses, e.g. `(marketing)`, groups routes for organizational purposes — applying a shared layout to a set of routes, or just keeping the file tree tidy — **without** adding a segment to the URL.

`app/(marketing)/about/page.tsx` is still served at `/about`, not `/marketing/about`. It's also how you give different top-level route groups (e.g. a marketing site vs a dashboard) entirely separate root layouts.

Q19. What are parallel routes, and when would you use them?

ADVANCED

Parallel routes let you render **multiple independent pages in the same layout simultaneously**, each in its own "slot," using the `@folderName` convention. Each slot has its own loading/error state and can be updated independently (e.g. navigating within one slot doesn't affect the other). Common use case: a dashboard with an `@analytics` slot and a `@team` slot rendered side-by-side, or a modal that needs to render alongside the page behind it.

PLAINTEXT

```
app/dashboard/  
  layout.tsx           // receives both slots as props  
  @analytics/  
    page.tsx  
  @team/  
    page.tsx  
  page.tsx
```

Q20. What are intercepting routes, and what's a typical use case?

ADVANCED

Intercepting routes let you load a different route within the current layout instead of the full route it points to — commonly used for showing a photo in a **modal** when navigated to via client-side navigation, while still showing the full standalone page if the URL is loaded directly (e.g. a hard refresh or shared link). The convention uses `(.)`, `(..)`, `(..)(..)`, or `(...)` prefixes to indicate how many segment levels up to match from.

PLAINTEXT

```
app/feed/
  page.tsx
  photo/[id]/page.tsx           // the real, full-page route
  (.)photo/[id]/page.tsx       // intercepted version, rendered as a modal
```

`(.)` intercepts a route at the same level — here, clicking a photo from the feed opens `(.)photo/[id]` as a modal, while visiting `/feed/photo/1` directly renders the real `photo/[id]/page.tsx`.

Q21.

What's the difference between `page.tsx`, `layout.tsx`, and `template.tsx`?

INTERMEDIATE

- `page.tsx` — the unique UI for a route segment; makes the segment publicly accessible.
- `layout.tsx` — shared UI that **wraps** a segment and its children, and **persists** across navigations (state isn't reset, it doesn't re-render on every navigation).
- `template.tsx` — like a layout, but creates a **new instance** on every navigation (state resets, effects re-run) — useful for enter/exit animations or resetting local state per-navigation.

Q22. How do you create a custom 404 page in the App Router?

INTERMEDIATE

TSX

```
// app/not-found.tsx - used for unmatched routes app-wide
export default function NotFound() {
  return (
    <div>
      <h2>Page not found</h2>
      <p>We could not find the page you were looking for.</p>
    </div>
  );
}

// Trigger it manually from anywhere in a Server Component:
import { notFound } from 'next/navigation';

async function ProductPage({ id }: { id: string }) {
  const product = await getProduct(id);
  if (!product) notFound();
  return <h1>{product.name}</h1>;
}
```

Q23. How do you implement redirects and rewrites in Next.js?

INTERMEDIATE

Redirects send the browser a new URL (visible in the address bar); **rewrites** serve different content for a URL while keeping the original URL displayed. Both can be configured statically in `next.config.js`, or dynamically at request time via `redirect()` (Server Components/Actions) or `NextResponse.redirect()` (Middleware/Route Handlers).

JAVASCRIPT

```
// next.config.js
module.exports = {
  async redirects() {
    return [{ source: '/old-blog/:slug', destination: '/blog/:slug', p
  },
  async rewrites() {
    return [{ source: '/api-proxy/:path*', destination: 'https://api.e
  },
};
```

4 Rendering Strategies

SSR, SSG, ISR, and CSR — what runs where, and when to reach for each.

Q24.

What is Server-Side Rendering (SSR), and when should you use it?

BEGINNER

SSR renders the page to HTML on the server for **every request**, using fresh data each time. Use it for pages whose content must always be current and is specific to the request (a logged-in user's dashboard, a page reflecting real-time inventory). The tradeoff is a slower response than a cached/static page, since the server does the work on every hit.

Q25.

What is Static Site Generation (SSG), and when should you use it?

BEGINNER

SSG renders the page to HTML once, **at build time**, and serves the same prebuilt file to every visitor (typically from a CDN). It's the fastest possible response and the cheapest to serve, but the content is only as fresh as the last build. Ideal for marketing pages, documentation, and blog posts that don't change per-request.

Q26.**What is Incremental Static Regeneration (ISR), and how does revalidate work?****INTERMEDIATE**

ISR lets a statically-generated page be **regenerated in the background** after it goes stale, without a full rebuild of the whole site. You set a revalidation interval; the first request after that window still gets the (stale) cached page instantly, but Next.js triggers a regeneration in the background and swaps in the fresh version for subsequent requests ("stale-while-revalidate"). This gives near-static performance with periodically fresh data.

TSX

```
// Revalidate this page's data at most once every 60 seconds
async function getPosts() {
  const res = await fetch('https://api.example.com/posts', {
    next: { revalidate: 60 },
  });
  return res.json();
}
```

Q27.**What is Client-Side Rendering (CSR), and when is it still the right choice in Next.js?****BEGINNER**

CSR renders in the browser after JS loads — the same model as a traditional SPA. Even in Next.js, this is appropriate for highly interactive, user-specific UI that doesn't need SEO or a fast first paint of real content: a live dashboard with frequent polling, an in-app settings panel, or any component that only makes sense after user interaction. You get this by fetching inside a `"use client"` component with `useEffect` / a data-fetching library (SWR, React Query) instead of fetching in a Server Component.

Q28. Compare SSR, SSG, ISR, and CSR — how do you choose?

INTERMEDIATE

- **SSG** — content is the same for everyone and rarely changes → marketing pages, docs.
- **ISR** — content is the same for everyone but changes periodically → blogs, product catalogs.
- **SSR** — content must be fresh and/or is personalized per request → user dashboards, search results.
- **CSR** — content is highly interactive, user-specific, and doesn't need to be indexed or shown before JS loads → in-app widgets, live data panels.

In the App Router, this is largely driven implicitly by *how* you fetch data (see Q31) rather than a function you export, unlike the Pages Router where you explicitly chose `getStaticProps` vs `getServerSideProps`.

Q29. What is streaming SSR, and how does Next.js implement it?

ADVANCED

Instead of waiting for the **entire** page's data to be ready before sending any HTML, streaming SSR sends the parts of the page that are ready immediately and streams in the slower parts as they finish — powered by React's `<Suspense>` boundaries. Next.js implements this natively in the App Router: wrap a slow-loading section in `<Suspense fallback={...}>`, and the rest of the page renders and becomes interactive without waiting on it.

TSX

```
import { Suspense } from 'react';

export default function DashboardPage() {
  return (
    <div>
      <Header />
      <Suspense fallback={<AnalyticsSkeleton />}>
        <SlowAnalyticsPanel /> /* streams in once its data resolves */
      </Suspense>
    </div>
  );
}
```

Q30.**What is `generateStaticParams`, and how does it relate to `getStaticPaths`?****INTERMEDIATE**

`generateStaticParams` is the App Router's equivalent of the Pages Router's `getStaticPaths` — it tells Next.js which dynamic route values to prerender at build time.

TSX

```
// app/blog/[slug]/page.tsx
export async function generateStaticParams() {
  const posts = await getAllPosts();
  return posts.map((post) => ({ slug: post.slug }));
}

export default async function BlogPost({ params }: { params: Promise<{
  slug: string;
}> }) {
  const { slug } = await params;
  const post = await getPost(slug);
  return <article>{post.content}</article>;
}
```

Q31.**How does the App Router decide whether a route is rendered statically or dynamically?****ADVANCED**

Next.js infers it automatically from what the route *does*, rather than a config flag you set explicitly. A route stays **static** (prerendered) unless it uses something that requires request-time information — reading cookies/headers, using `searchParams`, calling `fetch()` with `{ cache: 'no-store' }` or a short `revalidate`, or explicitly setting `export const dynamic = 'force-dynamic'`. As soon as any of those appear, the whole route (and everything above it that depends on it) is rendered dynamically per-request.

Q32. What is Partial Prerendering (PPR)? **ADVANCED**

PPR is a rendering model that combines static and dynamic rendering in the **same page**: the static "shell" (everything not dependent on request-time data) is prerendered and served instantly from the edge, while dynamic parts wrapped in `<Suspense>` are streamed in afterward, rendered per-request. This removes the old all-or-nothing choice where a single dynamic API call anywhere on a page forced the entire page to become dynamic — you get the speed of static delivery for the shell and the freshness of dynamic rendering only where it's actually needed.

5 Server & Client Components

Q33. What are React Server Components (RSC)? INTERMEDIATE

Server Components render **entirely on the server** (at build time or request time) and send only the resulting output to the browser — their component code, and any libraries they use, never ship to the client at all. This is the App Router's default for every component. It reduces client bundle size significantly and lets components directly access server-only resources (databases, the file system, secret API keys) without an extra API layer.

Q34.

Server Components vs Client Components — what's the actual difference?

INTERMEDIATE

- **Server Components** — render on the server only; can be `async` and fetch data directly; cannot use hooks like `useState` / `useEffect` or browser APIs; ship no JS to the client.
- **Client Components** — marked with `"use client"`; render on the server for the initial HTML too, then **hydrate** and can re-render in the browser; required for interactivity (state, effects, event handlers, browser-only APIs).

The default in `app/` is Server Component — you opt *into* Client Components, not out of them.

Q35. When do you need the "use client" directive? BEGINNER

Add `"use client"` at the top of a file whenever that component (or anything it directly uses) needs: React state or effects (`useState`, `useEffect`, `useReducer`), event handlers (`onClick`, `onChange`), browser-only APIs (`window`, `localStorage`), or third-party libraries that themselves rely on client-side React features.

TSX

```
'use client';

import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount((c) => c + 1)}>Count: {count}</button>
}
```

Q36.

Can Server Components import Client Components, and vice versa?

INTERMEDIATE

A Server Component **can** import and render a Client Component — this is the normal way to add an interactive island inside otherwise-static server-rendered content. A Client Component **cannot** directly import a Server Component (once you're in client territory, everything you import is bundled and treated as client code) — but it **can** receive an already-rendered Server Component as a `children` or other prop passed down from a parent Server Component (see Q39).

Q37.**How do Server and Client Components communicate, and what are the serialization limits?****ADVANCED**

Data flows from Server to Client Components as **props**, which must be **serializable** — plain objects, arrays, strings, numbers, booleans, and Dates are fine, but functions, class instances, Symbols, and React context values generally cannot be passed as props from server to client (functions can't be serialized across the server/client boundary, except Server Actions, which Next.js specially serializes as callable references). Client Components communicate back up to the server not via props, but by calling Server Actions or hitting Route Handlers.

Q38. What can't you do inside a Server Component?**INTERMEDIATE**

- No React state/effect hooks (`useState` , `useEffect` , `useContext` for client-side context).
- No event handlers (`onClick` , etc.) — they can't be serialized to the client.
- No browser-only APIs (`window` , `document` , `localStorage`).
- No React class component lifecycle methods requiring the client runtime.

Anything on this list means the component (or the specific interactive part of it) needs to be a Client Component instead.

Q39.**What is the "children as a prop" pattern for composing Server and Client Components?****ADVANCED**

Since a Client Component can't *import* a Server Component, but it can render whatever's passed to it as `children`, you can keep a Server Component's server-only benefits (like a heavy data-fetching child) while still using a Client Component as a wrapper for interactivity (like a modal or a tab container) around it.

TSX

```
// ClientWrapper.tsx
'use client';
export default function ClientWrapper({ children }: { children: React.ReactNode }) {
  const [open, setOpen] = useState(false);
  return (
    <div>
      <button onClick={() => setOpen(!open)}>Toggle</button>
      {open && children} { /* renders the Server Component passed in below */ }
    </div>
  );
}

// page.tsx (Server Component)
export default async function Page() {
  const data = await getServerOnlyData();
  return (
    <ClientWrapper>
      <ServerRenderedContent data={data} />
    </ClientWrapper>
  );
}
```

Q40. Why does using Server Components reduce client bundle size?**INTERMEDIATE**

Because a Server Component's code — and any library it imports (a Markdown parser, a date-formatting library, a database client) — runs and stays on the server; only the resulting rendered output is sent to the browser. In the Pages Router (or a plain SPA), that same component and every dependency it uses would be bundled and shipped as JavaScript the client has to download, parse, and execute, even though the output never changes based on client-side state. Moving non-interactive UI to Server Components directly shrinks what the browser has to load.

6 Data Fetching & Caching

Arguably the most interview-tested App Router topic — the four layered caches.

Q41. How do you fetch data in a Server Component? BEGINNER

Just make the component `async` and `await` your data directly — no `useEffect`, no loading-state boilerplate, since the component doesn't render until the data is ready (or streams via Suspense).

TSX

```
export default async function PostsPage() {
  const res = await fetch('https://api.example.com/posts');
  const posts = await res.json();
  return (
    <ul>
      {posts.map((post) => <li key={post.id}>{post.title}</li>)}
    </ul>
  );
}
```

Q42.**How does Next.js extend the `fetch()` API, and what do the caching options mean?****INTERMEDIATE**

Next.js patches the native `fetch` so every call can opt into its caching system via extra options:

TSX

```
fetch('https://api.example.com/data', {  
  cache: 'no-store', // or 'force-cache'  
  next: { revalidate: 3600, tags: ['products'] },  
});
```

- `cache: 'force-cache'` (default) — cache indefinitely, like SSG.
- `cache: 'no-store'` — never cache, fetch fresh on every request, like SSR.
- `next: { revalidate: N }` — cache and revalidate after N seconds, like ISR.
- `next: { tags: [...] }` — tag this fetch so it can be invalidated on demand later (see Q47).

Q43. What is the Data Cache? **ADVANCED**

A persistent, server-side cache (surviving across requests and deployments, in supported environments) that stores the results of individual `fetch()` calls based on their caching options. It's what powers ISR-style revalidation at the level of a single fetch, rather than the whole page — so you can cache one API call for an hour while another on the same page always fetches fresh.

Q44. What is Request Memoization? INTERMEDIATE

Within a **single render pass**, if multiple components call `fetch()` with the exact same URL and options, Next.js automatically deduplicates them into a single network request — the result is memoized and reused for the rest of that render. This means you don't need to worry about prop-drilling data down through many components just to avoid duplicate fetches; each component can independently fetch what it needs. This memoization is cleared between separate requests, unlike the Data Cache.

Q45. What is the Full Route Cache? ADVANCED

At build time, Next.js renders static routes to HTML and an RSC payload and stores that **rendered output** on the server — this is the Full Route Cache. It persists across requests until the route is revalidated or redeployed. This is distinct from the Data Cache (which caches the raw fetch results that fed into the render) — the Full Route Cache stores the finished render itself, so a fully static route can be served without even re-running the component tree.

Q46. What is the Router Cache (client-side)? ADVANCED

An in-memory, client-side cache in the browser that stores the RSC payload for routes the user has already visited or Next.js has prefetched (e.g. via `<Link>`). It's what makes back/forward navigation and revisiting a link feel instant — no server round-trip needed. It's per-browser-session, cleared on a full page reload, and is separate from both server-side caches above.

Q47.**How do you invalidate cached data on demand with `revalidatePath` and `revalidateTag`?****INTERMEDIATE**

Both let you purge cached data outside of the normal time-based `revalidate` window — typically called from a Server Action or a webhook-triggered Route Handler right after a mutation, so users see fresh data immediately instead of waiting for the next scheduled revalidation.

TSX

```
'use server';
import { revalidatePath, revalidateTag } from 'next/cache';

export async function publishPost(id: string) {
  await db.posts.update(id, { published: true });
  revalidateTag('posts');           // invalidate every fetch tagged 'posts'
  revalidatePath('/blog');         // invalidate the cached /blog route
}
```

Q48.**How do you avoid data-fetching waterfalls in Server Components?****ADVANCED**

A waterfall happens when independent fetches are awaited sequentially, each one blocking the start of the next, even though they don't depend on each other. Fix it by kicking off the requests in parallel with `Promise.all` (since Request Memoization means calling `fetch()` early doesn't cost an extra request) instead of awaiting them one at a time.

TSX

```
// Slow: sequential - second fetch doesn't start until the first finishes  
async function Page() {  
  const user = await getUser();  
  const posts = await getPosts(); // waits unnecessarily  
  return <Profile user={user} posts={posts} />;  
}  
  
// Fast: parallel - both requests fire immediately  
async function PageFast() {  
  const [user, posts] = await Promise.all([getUser(), getPosts()]);  
  return <Profile user={user} posts={posts} />;  
}
```

Q49.**How did data fetching work in the Pages Router, and how does it map to the App Router?****INTERMEDIATE**

- `getStaticProps` (build-time, cached) → an `async` Server Component using `fetch()` with the default `force-cache`.
- `getServerSideProps` (every request) → an `async` Server Component using `fetch(url, { cache: 'no-store' })`.
- `getStaticPaths` → `generateStaticParams`.
- `getInitialProps` (legacy, ran on both server and client) → generally avoided now; replaced by Server Components entirely.

7 Server Actions & Mutations

Q50. What are Server Actions?

INTERMEDIATE

Server Actions are async functions marked with `"use server"` that run **only on the server** but can be called directly from Client (or Server) Components as if they were local functions — no manually-written API route or client-side fetch call needed. Next.js handles serializing the call, sending it to the server, and returning the result, including automatic progressive enhancement when used directly as a `<form action={...}>`.

Q51. How do you define and use a Server Action with a form?

INTERMEDIATE

TSX

```
// app/actions.ts
'use server';

export async function createTodo(formData: FormData) {
  const title = formData.get('title') as string;
  await db.todos.create({ title });
}

// app/page.tsx (Server Component)
import { createTodo } from './actions';

export default function Page() {
  return (
    <form action={createTodo}>
      <input name="title" />
      <button type="submit">Add</button>
    </form>
  );
}
```

Q52.

How do you call a Server Action from a Client Component's event handler?

INTERMEDIATE

TSX

```
'use client';
import { deleteTodo } from './actions';

export default function DeleteButton({ id }: { id: string }) {
  return (
    <button onClick={async () => { await deleteTodo(id); }}>
      Delete
    </button>
  );
}
```

Q53.

How do Server Actions handle cache revalidation after a mutation?

INTERMEDIATE

A Server Action is a normal server-side function, so after performing the mutation (a DB write, for example) it should explicitly call `revalidatePath()` or `revalidateTag()` before returning — this is not automatic. Without it, cached routes/fetches that display the now-stale data won't refresh until their normal revalidation window passes.

Q54. What security considerations apply to Server Actions?

ADVANCED

- A Server Action is effectively a **public HTTP endpoint** once deployed — Next.js generates a unique, unguessable ID per action, but it's still callable by anyone who can reach your server, not just your UI. Always re-validate authentication/authorization **inside** the action itself, never rely on the fact that "only my button calls this."
- Re-validate and sanitize all input arguments — treat them exactly like an API request body.
- Don't pass sensitive data as a "closed-over" variable into a Server Action defined inline in a way that could leak it to the client bundle — keep secrets read on the server side, inside the action.

Q55. What do `useFormStatus` and `useActionState` do? INTERMEDIATE

`useFormStatus` (from `react-dom`) reads the pending state of the nearest parent `<form>`'s submission — commonly used to disable a submit button or show a spinner while a Server Action is running. `useActionState` (formerly `useFormState`) wraps a Server Action to also track and return its latest result/validation errors back into the component's state, enabling patterns like showing a field-level error message after a failed submission.

TSX

```
'use client';
import { useFormStatus } from 'react-dom';

function SubmitButton() {
  const { pending } = useFormStatus();
  return <button disabled={pending}>{pending ? 'Saving...' : 'Save'}</button>
}
```

8 Route Handlers (API Routes)

Q56. What are Route Handlers in the App Router? BEGINNER

Route Handlers let you build custom API endpoints inside `app/` using a `route.ts` file, exporting a function named after the HTTP method it handles (`GET`, `POST`, etc.), built on the standard Web `Request` / `Response` APIs.

TSX

```
// app/api/hello/route.ts
export async function GET() {
  return Response.json({ message: 'Hello' });
}
```

Q57.

How do Route Handlers differ from the old Pages Router API Routes?

INTERMEDIATE

Functionally similar (both build backend endpoints colocated with the app), but Route Handlers use the standard Web `Request` / `Response` objects instead of the Node-specific `req` / `res` objects from `pages/api`, one exported function per HTTP verb instead of a single handler checking `req.method`, and they can run on either the Node.js or Edge runtime (configurable), while Pages API Routes always run on Node.js.

Q58.

How do you handle multiple HTTP methods in one Route Handler file?

INTERMEDIATE

TSX

```
// app/api/todos/route.ts
export async function GET() {
  const todos = await db.todos.findAll();
  return Response.json(todos);
}

export async function POST(request: Request) {
  const body = await request.json();
  const todo = await db.todos.create(body);
  return Response.json(todo, { status: 201 });
}
```

Q59.**How do you read query parameters and the request body in a Route Handler?**

INTERMEDIATE

TSX

```
// app/api/search/route.ts
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const query = searchParams.get('q');
  return Response.json({ results: await search(query) });
}

// Dynamic segment: app/api/users/[id]/route.ts
export async function DELETE(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  await db.users.delete(id);
  return new Response(null, { status: 204 });
}
```

Q60. Can Route Handler responses be cached?

ADVANCED

A `GET` Route Handler is **static/cacheable by default** if it doesn't use dynamic APIs (like reading cookies/headers or `request.url` in certain ways) — it can be included in the Full Route Cache just like a page. Any other HTTP method (`POST`, `PUT`, `DELETE`, etc.) is **never** cached, since they're assumed to be mutations. You can force a `GET` handler to always be dynamic with `export const dynamic = 'force-dynamic'`.

Q61.**When should you use a Route Handler instead of a Server Action?****INTERMEDIATE**

Use a **Server Action** for mutations triggered directly from your own UI (forms, buttons) where you want the ergonomics of calling a function directly with no separate API layer. Use a **Route Handler** when you need a real, addressable HTTP endpoint — for third-party webhooks, when the client is a mobile app or external service (not your Next.js UI), when you need fine control over response headers/status codes/caching, or when exposing a public API contract that isn't tied to a specific form submission.

9 Middleware

Q62. What is Next.js Middleware, and where does it run?

INTERMEDIATE

Middleware is code defined in a single `middleware.ts` file at the project root that runs **before** a request is completed — before a route is matched to a page, Route Handler, or static asset. It runs on the **Edge Runtime** (a lightweight, fast-starting JS runtime, not full Node.js) so it executes close to the user with minimal latency, on every request that matches its configured paths.

TYPESCRIPT

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const country = request.geo?.country ?? 'US';
  const response = NextResponse.next();
  response.headers.set('x-user-country', country);
  return response;
}
```

Q63. What are common use cases for Middleware? INTERMEDIATE

- **Authentication guards** — redirect unauthenticated users away from protected routes before the page even renders.
- **Redirects/rewrites based on request data** — locale detection, A/B testing, feature flags.
- **Geolocation-based logic** — serving different content per region.
- **Header/cookie manipulation** — attaching a request ID, setting security headers.
- **Bot detection / basic rate limiting** at the edge, before hitting your origin.

Q64. How do you restrict Middleware to specific paths? INTERMEDIATE

TYPESCRIPT

```
export const config = {  
  matcher: ['/dashboard/:path*', '/settings/:path*'],  
};
```

Q65. What are the limitations of Middleware? ADVANCED

Because it runs on the Edge Runtime, Middleware can't use Node.js-only APIs (the `fs` module, most native Node packages, TCP database drivers like the standard `pg` client). It should also be kept lightweight — it runs on **every** matched request before any caching kicks in, so slow logic there (e.g. an uncached database call) adds latency to the entire site. Prefer HTTP-based calls (e.g. to an edge-compatible database or a REST API) over native Node database drivers inside Middleware.

Q66.

Example: protect a set of routes with authentication using Middleware.

ADVANCED**TYPESCRIPT**

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const token = request.cookies.get('session')?.value;

  if (!token) {
    const loginUrl = new URL('/login', request.url);
    loginUrl.searchParams.set('from', request.nextUrl.pathname);
    return NextResponse.redirect(loginUrl);
  }

  return NextResponse.next();
}

export const config = { matcher: ['/dashboard/:path*'] };
```

10 Layouts, Loading & Streaming

Q67. What is the root layout, and why is it required? BEGINNER

Every App Router project must have a top-level `app/layout.tsx` that renders the `<html>` and `<body>` tags — Next.js doesn't inject these for you, since the App Router doesn't have an equivalent of the Pages Router's automatic HTML document. It wraps every route in the app and is the right place for global providers, fonts, and analytics scripts.

TSX

```
// app/layout.tsx
export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

Q68. How do nested layouts persist state across navigations?

INTERMEDIATE

When you navigate between two routes that share a layout, Next.js only re-renders the parts of the tree **below** the shared layout — the layout itself is not unmounted and remounted, so its local React state, scroll position, and any running effects (like a video still playing in a persistent sidebar) are preserved. This is a deliberate difference from `template.tsx`, which does reset on every navigation.

Q69. How does loading.tsx work? INTERMEDIATE

A `loading.tsx` file in a route segment automatically wraps that segment's `page.tsx` in a `<Suspense>` boundary, using the file's export as the fallback UI. Next.js shows it immediately on navigation while the async Server Component fetches its data, then swaps in the real content once ready — no manual Suspense wiring needed for the top-level route.

TSX

```
// app/dashboard/loading.tsx
export default function Loading() {
  return <p>Loading dashboard...</p>;
}
```

Q70. How does `error.tsx` work as a per-segment error boundary?

INTERMEDIATE

An `error.tsx` file automatically wraps its segment in a React error boundary. It must be a Client Component, receives the thrown `error` and a `reset()` function as props, and catches errors from anywhere in that segment's rendering (including nested Server Components) without taking down the rest of the app — the layout above it keeps working.

TSX

```
// app/dashboard/error.tsx
'use client';

export default function Error({
  error,
  reset,
}: {
  error: Error;
  reset: () => void;
}) {
  return (
    <div>
      <p>Something went wrong: {error.message}</p>
      <button onClick={() => reset()}>Try again</button>
    </div>
  );
}
```

Q71. What is `global-error.tsx`, and how is it different from `error.tsx`?

ADVANCED

`error.tsx` catches errors within its route segment but assumes the root layout above it still rendered successfully. `global-error.tsx` is the fallback for errors in the **root layout itself** — since it replaces the root layout entirely when active, it must render its own `<html>` and `<body>` tags. It's rarely triggered in practice, but required to handle the edge case of the root layout failing.

Q72.**How does streaming with Suspense improve perceived performance?****INTERMEDIATE**

Without streaming, a page's Time to First Byte is gated on its **slowest** data dependency — nothing renders until everything is ready. With Suspense boundaries around slow sections, the fast parts of the page (header, nav, above-the-fold content) render and become visible almost immediately, while the slow section shows a skeleton and streams in moments later. Users perceive the page as loading much faster even though the total data-fetching time hasn't changed — only the order in which things become visible has.

Q73.**What's the difference between `not-found.tsx` and calling `notFound()`?****INTERMEDIATE**

`not-found.tsx` is the UI file rendered whenever a route can't be matched at all, or when the `notFound()` function (imported from `next/navigation`) is called inside a Server Component — e.g. after checking that a requested database record doesn't exist. Calling `notFound()` immediately halts rendering of that segment and tells Next.js to render the nearest `not-found.tsx` boundary, responding with an HTTP 404 status.

11 Image, Font & Script Optimization

Q74.

What optimizations does next/image provide over a plain `<img>` tag?

BEGINNER

- Automatic **resizing and serving in modern formats** (WebP/AVIF) based on the requesting device.
- **Lazy loading** by default — images outside the viewport aren't fetched until they're about to scroll into view.
- **Prevents layout shift** by reserving the correct space using the required `width` / `height` (or `fill`), improving Cumulative Layout Shift (CLS).
- Built-in support for `priority` on above-the-fold images to improve Largest Contentful Paint (LCP).

Q75. How do you handle responsive images with next/image?

INTERMEDIATE

TSX

```
import Image from 'next/image';

export default function Hero() {
  return (
    <Image
      src="/hero.jpg"
      alt="Product hero shot"
      fill
      sizes="(max-width: 768px) 100vw, 50vw"
      style={{ objectFit: 'cover' }}
      priority
    />
  );
}
```

Q76.**How does next/font eliminate layout shift and external font requests?****INTERMEDIATE**

`next/font` downloads and **self-hosts** font files (including Google Fonts) at build time, so the browser never makes a request to an external font CDN — improving privacy and removing that network round-trip. It also automatically applies `size-adjust` CSS to closely match the fallback system font's metrics, minimizing the layout shift that occurs when the real font finishes loading and swaps in.

TSX

```
import { Inter } from 'next/font/google';

const inter = Inter({ subsets: ['latin'], display: 'swap' });

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={inter.className}>
      <body>{children}</body>
    </html>
  );
}
```

Q77. What loading strategies does next/script offer? INTERMEDIATE

- `beforeInteractive` — loads and executes before any page JS becomes interactive; use only for scripts a bot/polyfill critically needs before hydration.
- `afterInteractive` (default) — loads soon after the page becomes interactive; good for most analytics/tag-manager scripts.
- `lazyOnload` — loads during idle browser time, lowest priority; good for chat widgets or non-critical third-party scripts.
- `worker` (experimental) — offloads the script to a web worker, off the main thread entirely.

TSX

```
import Script from 'next/script';  
  
<Script src="https://analytics.example.com/script.js" strategy="afterI
```

Q78. How would you optimize a large hero image to improve LCP?INTERMEDIATE

- Render it with `next/image` and set the `priority` prop so it's preloaded and skips lazy loading.
- Serve it already sized/cropped appropriately rather than relying entirely on runtime resizing for a huge source file.
- Prefer modern formats (AVIF/WebP), which `next/image` handles automatically when the browser supports them.
- Avoid placing render-blocking JS/CSS above it, and make sure the hero section itself isn't gated behind a slow data fetch (keep it static or in a fast-resolving Server Component).

Q79.**What's a common next/image pitfall when using remote/external images?****INTERMEDIATE**

By default, `next/image` refuses to optimize images from domains it doesn't know about, for security (to prevent abuse of the optimization endpoint as an open proxy). You must explicitly allow-list any external image host in `next.config.js`, or the image will fail to load.

JAVASCRIPT

```
// next.config.js
module.exports = {
  images: {
    remotePatterns: [{ protocol: 'https', hostname: 'cdn.example.com'
  },
};
```

12 SEO & Metadata

Q80. How do you set static metadata for a page? BEGINNER

TSX

```
// app/about/page.tsx
import type { Metadata } from 'next';

export const metadata: Metadata = {
  title: 'About Us',
  description: 'Learn more about our company and mission.',
};

export default function AboutPage() {
  return <h1>About</h1>;
}
```

Q81.

How do you generate dynamic metadata per page (e.g. a blog post's title)?

INTERMEDIATE**TSX**

```
// app/blog/[slug]/page.tsx
import type { Metadata } from 'next';

export async function generateMetadata({
  params,
}: {
  params: Promise<{ slug: string }>;
}): Promise<Metadata> {
  const { slug } = await params;
  const post = await getPost(slug);
  return {
    title: post.title,
    description: post.excerpt,
    openGraph: { images: [post.coverImage] },
  };
}
```

Q82. How do you generate a sitemap in Next.js?**INTERMEDIATE**

Add a `sitemap.ts` file under `app/` that exports a default function returning an array of URL entries; Next.js automatically serves it at `/sitemap.xml` with the correct XML formatting. See Q117 for a full dynamic implementation pulling URLs from a database/CMS.

Q83. How do you generate a robots.txt file?

BEGINNER

TYPESCRIPT

```
// app/robots.ts
import type { MetadataRoute } from 'next';

export default function robots(): MetadataRoute.Robots {
  return {
    rules: { userAgent: '*', allow: '/', disallow: '/admin/' },
    sitemap: 'https://example.com/sitemap.xml',
  };
}
```

Q84. How do you set up Open Graph and Twitter card metadata?

INTERMEDIATE

TSX

```
export const metadata: Metadata = {
  openGraph: {
    title: 'My Product',
    description: 'The best product for your needs.',
    images: ['/og-image.png'],
    type: 'website',
  },
  twitter: {
    card: 'summary_large_image',
    title: 'My Product',
    images: ['/og-image.png'],
  },
};
```

Q85.**What is metadataBase, and why does it matter for canonical/OG URLs?****INTERMEDIATE**

Relative URLs used inside `openGraph.images`, canonical links, and other metadata fields need an absolute base to resolve against when rendered into `<meta>` tags. Setting `metadataBase` once in the root layout's metadata lets every nested page use relative paths (e.g. `/og-image.png`) and have Next.js automatically resolve them to full URLs (e.g. `https://example.com/og-image.png`) — without it, Next.js falls back to a default and warns in development.

13 Styling Solutions

Q86. How do CSS Modules work in Next.js? BEGINNER

Any file named `*.module.css` is automatically scoped — Next.js generates unique class names at build time so styles never leak between components, no extra configuration required.

TSX

```
// Button.module.css
// .primary { background: blue; color: white; }

import styles from './Button.module.css';
export default function Button() {
  return <button className={styles.primary}>Click</button>;
}
```

Q87. Where can global CSS be imported in the App Router? BEGINNER

BEGINNER

Global stylesheets can only be imported in the **root layout** (`app/layout.tsx`) or another top-level layout — importing global CSS inside a regular page or nested component is not allowed, since global styles aren't scoped and could unpredictably affect the rest of the app depending on render order.

Q88. How do you set up Tailwind CSS in a Next.js project? BEGINNER

`create-next-app` offers a Tailwind option out of the box. Manually, it's: install `tailwindcss` and its PostCSS plugin, generate a config that scans your `app/` / `components/` directories for class names, and import Tailwind's base/components/utilities layers in your root global CSS file, which is then imported once in the root layout.

Q89.**What challenges do CSS-in-JS libraries (styled-components, Emotion) face with Server Components?****ADVANCED**

Traditional CSS-in-JS libraries generate styles at runtime **in the browser** or rely on a React context provider to collect styles during rendering for SSR — both approaches assume a Client Component render model. In a Server Component (which never runs in the browser and has no React context in the traditional sense), that mechanism breaks. These libraries need special App Router integration (a registered style-sheet-collecting component wrapped around the root layout) to work at all, and any component actually using the CSS-in-JS API must be a Client Component. For this reason, many App Router projects prefer CSS Modules, Tailwind, or vanilla-extract, which don't have a runtime and work naturally with Server Components.

Q90.**How would you approach dark mode with SSR, avoiding a flash of the wrong theme?****INTERMEDIATE**

The core problem: the server doesn't know the user's preferred theme until the browser tells it, so a naive implementation renders one theme's HTML, then flashes to the correct theme once client JS runs. The standard fix is the **next-themes** library, which injects a small blocking inline script before hydration that reads the stored preference (or system preference) and sets the correct class/attribute on `<html>` before the page paints — see the full implementation in Q118.

14 Performance Optimization

Q91. Which Core Web Vitals does Next.js help most with, and how?

INTERMEDIATE

- **LCP (Largest Contentful Paint)** — improved by server rendering/static generation (content is in the initial HTML, not waiting on client JS) and `next/image`'s `priority` loading.
- **CLS (Cumulative Layout Shift)** — improved by `next/image`'s enforced dimensions and `next/font`'s fallback-metric matching, preventing content jump as images/fonts load.
- **INP (Interaction to Next Paint)** — improved by shipping less client JS overall (Server Components by default) and code-splitting, so the main thread is less busy/blocked when the user interacts.

Q92. How do you code-split a component with next/dynamic?

INTERMEDIATE

Use `next/dynamic` to lazily load a component (and its own JS chunk) only when it's actually needed — useful for large, rarely-used components (a rich text editor, a charting library) that shouldn't be in the initial bundle.

TSX

```
import dynamic from 'next/dynamic';

const HeavyChart = dynamic(() => import('../components/HeavyChart'), {
  loading: () => <p>Loading chart...</p>,
  ssr: false, // skip server rendering for a client-only-heavy widget
});
```

Q93. How do you analyze what's bloating your Next.js bundle?

INTERMEDIATE

The `@next/bundle-analyzer` plugin wraps your Next.js config and, when enabled, generates an interactive treemap visualization of every module in your client and server bundles after `next build` — making it easy to spot an unexpectedly large dependency (e.g. a full date library imported for one function) and swap it for a lighter alternative or a more targeted import.

Q94.

How does `<Link>` prefetching work, and how does it improve navigation performance?

INTERMEDIATE

By default, when a `<Link>` component scrolls into the viewport, Next.js automatically prefetches that route's data/RSC payload in the background (for static routes, the full payload; for dynamic routes, just the shared layout shell). By the time the user actually clicks, the data is often already in the Router Cache, making the navigation feel instant instead of waiting for a fresh request.

Q95.

How do you avoid data-fetching waterfalls at the page/layout level, beyond a single component?

ADVANCED

A common hidden waterfall: a layout awaits its own data before rendering its children, which delays the child page's fetch from even *starting* until the layout's fetch finishes, even though they're unrelated. Fix by not blocking rendering of children on unrelated parent data — start independent fetches as early and in parallel as possible, and use Suspense boundaries so a slow parent-level fetch doesn't gate everything below it.

Q96.**What does React's `cache()` function do, and how is it used in Next.js?****ADVANCED**

`cache()` (from `react`) memoizes the result of a function for the duration of a single render pass — the same mechanism that powers Request Memoization for `fetch()`, but usable for **any** data source, like a direct database query, where automatic dedup doesn't apply.

TSX

```
import { cache } from 'react';

export const getUser = cache(async (id: string) => {
  return db.user.findUnique({ where: { id } });
});

// Calling getUser(id) from multiple components in the same render
// only hits the database once.
```

Q97.**Edge Runtime vs Node.js Runtime — what are the performance tradeoffs?****ADVANCED**

The **Edge Runtime** is a minimal JS runtime that starts up nearly instantly (no cold-start penalty of spinning up a full Node process) and runs physically closer to users across many edge locations, which lowers latency for simple, stateless logic — but it lacks most Node.js APIs and native modules. The **Node.js Runtime** gives full compatibility with the npm ecosystem (native database drivers, file system access, larger libraries) and is better suited for heavier compute, but has slower cold starts and typically runs from fewer regions. Choose Edge for latency-sensitive, lightweight logic (auth checks, redirects, simple personalization); choose Node.js when you need full library compatibility or heavier processing.

15 Authentication & Security

Q98. What are common authentication patterns in Next.js?

INTERMEDIATE

- **Auth.js (NextAuth)** — the most common library, handling OAuth providers, session/JWT strategies, and Route Handler wiring specifically for Next.js.
- **Custom session cookies** — issuing your own signed/encrypted session cookie after login, verified in Middleware or Server Components.
- **Third-party auth platforms** (Clerk, Auth0, Supabase Auth) — hosted auth with SDKs providing prebuilt Next.js integration (middleware helpers, server-side session readers).

Q99.

How do you protect a Server Component or route based on authentication?

INTERMEDIATE

TSX

```
// app/dashboard/page.tsx
import { redirect } from 'next/navigation';
import { getSession } from '@/lib/session';

export default async function DashboardPage() {
  const session = await getSession();
  if (!session) redirect('/login');

  return <h1>Welcome, {session.user.name}</h1>;
}
```

Q100. How do you protect a Server Action from unauthorized calls?

INTERMEDIATE

TSX

```
'use server';
import { getSession } from '@lib/session';

export async function deletePost(postId: string) {
  const session = await getSession();
  if (!session) throw new Error('Unauthorized');

  const post = await db.posts.findUnique({ where: { id: postId } });
  if (post.authorId !== session.user.id) throw new Error('Forbidden');

  await db.posts.delete({ where: { id: postId } });
}
```

Q101. How should you store session tokens securely in Next.js?

INTERMEDIATE

Store session tokens in an **httpOnly, Secure, SameSite** cookie rather than `localStorage` — `httpOnly` prevents JavaScript (and therefore XSS payloads) from reading it, `Secure` ensures it's only sent over HTTPS, and `SameSite=Lax/Strict` reduces CSRF exposure. Server Components and Route Handlers can read cookies directly via `cookies()` from `next/headers` without exposing the token to client-side JS at all.

Q102. Do Server Actions need CSRF protection? **ADVANCED**

Next.js applies built-in CSRF protections for Server Actions invoked via `<form action={...}>`: it verifies the request's `Origin` header matches the deploying host, rejecting cross-origin form submissions to a Server Action by default. That said, if a Server Action is also reachable in a way that bypasses this (e.g. exposed as a general-purpose endpoint another system calls), you're responsible for your own authorization/CSRF handling in that path — treat any Server Action performing a sensitive mutation as something to explicitly re-verify (see Q54/Q100), not just something implicitly safe because "only my form calls it."

Q103.

What does the `NEXT_PUBLIC_` environment variable prefix mean, and why does it matter for security?

BEGINNER

By default, environment variables are only available on the **server** — they're never bundled into client-side JS. Prefixing one with `NEXT_PUBLIC_` tells Next.js to inline its value into the client bundle at build time, making it publicly visible to anyone who inspects the deployed JS. Never put secrets (API keys, database URLs, signing secrets) behind this prefix — only genuinely public values (a public analytics ID, a public API base URL) should use it.

Q104. How do you set a Content Security Policy (CSP) in Next.js?

ADVANCED

CSP headers can be set in `next.config.js` for static values, or dynamically in Middleware when you need a per-request nonce (recommended for inline scripts, since a nonce prevents attackers from injecting their own inline `<script>` even if they find an XSS injection point).

TYPESCRIPT

```
// middleware.ts
import { NextResponse } from 'next/server';

export function middleware() {
  const nonce = crypto.randomUUID();
  const response = NextResponse.next();
  response.headers.set(
    'Content-Security-Policy',
    `script-src 'self' 'nonce-${nonce}'; object-src 'none';`
  );
  return response;
}
```

16 Deployment & Runtimes

Q105. What are the main ways to deploy a Next.js app? INTERMEDIATE

- **Vercel** — the framework's creator; zero-config deployment with first-class support for every feature (ISR, Edge Middleware, Image Optimization) out of the box.
- **Self-hosted Node.js server** — run `next build && next start` (or an `output: 'standalone'` build) behind your own infrastructure/container platform; you're responsible for wiring up equivalents of ISR's on-demand revalidation and CDN caching yourself.
- **Static export** (`output: 'export'`) — produces plain static HTML/CSS/JS with no Node.js server needed at all, deployable to any static host; only works if the app doesn't use features that require a server (SSR, Route Handlers with dynamic logic, Image Optimization's server component, Server Actions).

Q106. What is `output: 'standalone'`, and why would you use it?

INTERMEDIATE

It produces a minimal, self-contained build output that includes only the exact `node_modules` files actually needed to run the app in production (via dependency tracing), rather than requiring the full `node_modules` folder. This dramatically shrinks Docker image sizes and deployment artifacts when self-hosting.

JAVASCRIPT

```
// next.config.js
module.exports = { output: 'standalone' };
```

Q107.

How do you choose the runtime (Edge vs Node.js) for a specific route?

INTERMEDIATE**TYPESCRIPT**

```
// app/api/edge-example/route.ts
export const runtime = 'edge'; // or 'nodejs' (the default)

export async function GET() {
  return Response.json({ runtime: 'edge' });
}
```

Q108.

How do environment variables differ across .env files and environments?

BEGINNER

- `.env` — loaded in all environments (base defaults).
- `.env.local` — loaded in all environments, ignored by git; for personal/local secrets.
- `.env.development` / `.env.production` — loaded only when running in that specific environment (`next dev` vs a production build).
- Load order means more specific files override less specific ones. In hosted platforms (Vercel, etc.), environment variables are typically set through the platform's dashboard instead of committed `.env` files.

Q109.**How does ISR behave differently on Vercel vs a self-hosted Next.js server?****ADVANCED**

On Vercel, ISR is backed by their global edge network and durable storage — revalidated pages propagate globally and on-demand revalidation (`revalidatePath` / `revalidateTag`) works automatically across serverless invocations. Self-hosted, ISR relies on the Node.js server's local filesystem cache by default — meaning in a multi-instance/multi-container deployment behind a load balancer, each instance may hold a different cached version unless you configure a shared cache (e.g. Redis) yourself, since there's no built-in cross-instance cache invalidation like Vercel provides.

Q110. What are the limitations of static export (output: 'export')?**INTERMEDIATE**

Static export produces plain files with no server at request time, so it's incompatible with anything requiring server-side execution per request: `getServerSideProps` - equivalent dynamic Server Components, Route Handlers that do more than return static data, Server Actions, Middleware, ISR's background revalidation, and the default server-based Image Optimization (you'd need an external image loader instead). It's a good fit only for fully static sites/documentation, not for apps needing any server-side dynamic behavior.

17 Testing & Debugging

Q111. How do you unit test Next.js components?

INTERMEDIATE

Jest or **Vitest** combined with **React Testing Library** is the standard setup — render a component (Client Components test straightforwardly this way) and assert on the rendered output/behavior, mocking any Next-specific APIs (`next/navigation`'s `useRouter`, etc.) that the component depends on.

TSX

```
import { render, screen, fireEvent } from '@testing-library/react';
import Counter from './Counter';

test('increments the count on click', () => {
  render(<Counter />);
  fireEvent.click(screen.getByRole('button'));
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

Q112. How do you test async Server Components?

ADVANCED

Server Components are trickier to unit test directly since standard React Testing Library assumes a client render model, and an `async` component isn't a typical function component. A common approach: test the underlying data-fetching functions and business logic as plain async functions in isolation (they're just functions — no rendering needed), and rely on **integration/E2E tests** (Playwright hitting a real running app) to verify the full server-rendered output, rather than trying to force a Server Component through a unit-test renderer.

Q113. How do you write E2E tests for a Next.js app? INTERMEDIATE

TYPESCRIPT

```
import { test, expect } from '@playwright/test';

test('user can log in and see the dashboard', async ({ page }) => {
  await page.goto('/login');
  await page.fill('input[name="email"]', 'user@example.com');
  await page.fill('input[name="password"]', 'password');
  await page.click('button[type="submit"]');
  await expect(page).toHaveURL('/dashboard');
  await expect(page.getByText('Welcome')).toBeVisible();
});
```

Q114. How do you debug a hydration mismatch error? ADVANCED

Hydration mismatches happen when the server-rendered HTML doesn't match what the client would render on first pass. Common causes: using

`Date.now()` / `Math.random()` directly in render output, rendering something conditionally based on `typeof window !== 'undefined'`, browser extensions injecting DOM before hydration, or invalid HTML nesting (e.g. a `<div>` inside a `<p>`) that the browser silently "fixes" differently than React expects. React's dev-mode error message identifies the specific mismatched DOM node — start there, and move any genuinely client-only output into a `useEffect` so it only renders after hydration, or wrap it with `suppressHydrationWarning` only when the mismatch is truly expected and harmless (e.g. a rendered timestamp).

Q115.

What are some common Next.js build errors, and how do you diagnose them?

INTERMEDIATE

- **"You're importing a component that needs useState... but none of its parents are marked 'use client'"** — a Server Component is importing something that uses client-only hooks; add `"use client"` at the top of the offending file.
- **Dynamic server usage errors during static generation** — a route is trying to be static but calls `cookies()` / `headers()`; either mark it dynamic explicitly or restructure to avoid request-time APIs in a statically-generated path.
- **Module not found** for a Node-only package used inside Middleware or an Edge route — that package isn't Edge-compatible; switch the route to the Node.js runtime or find an edge-compatible alternative.
- **Environment variable undefined in the browser** — forgot the `NEXT_PUBLIC_` prefix on a variable actually needed client-side.

18 Practical Coding Challenges

Frequently asked "build this feature" implementation questions.

Q116.

Implement infinite scroll: a Server Component renders the first page, a Client Component with an IntersectionObserver loads more.

SENIOR

Pattern: render the first page of results on the server (fast initial paint, good SEO for that first page), then hand off to a Client Component that watches a sentinel element and calls a Server Action or Route Handler to fetch and append subsequent pages, tracking a cursor/offset in state.

TSX

```
// app/feed/page.tsx (Server Component)
import { getPosts } from '@/lib/posts';
import InfiniteFeed from './InfiniteFeed';

export default async function FeedPage() {
  const initialPosts = await getPosts({ cursor: null, limit: 10 });
  return <InfiniteFeed initialPosts={initialPosts.items} initialCursor:
}

// app/feed/InfiniteFeed.tsx (Client Component)
'use client';
import { useState, useRef, useEffect, useCallback } from 'react';
import { fetchMorePosts } from './actions';

export default function InfiniteFeed({ initialPosts, initialCursor })
  const [posts, setPosts] = useState(initialPosts);
  const [cursor, setCursor] = useState(initialCursor);
  const [loading, setLoading] = useState(false);
  const sentinelRef = useRef<HTMLDivElement>(null);

  const loadMore = useCallback(async () => {
    if (!cursor || loading) return;
    setLoading(true);
    const next = await fetchMorePosts(cursor);
    setPosts((prev) => [...prev, ...next.items]);
    setCursor(next.nextCursor);
    setLoading(false);
  }, [cursor, loading]);

  useEffect(() => {
    const observer = new IntersectionObserver(
      (entries) => { if (entries[0].isIntersecting) loadMore(); },
      { rootMargin: '200px' }
    );
    if (sentinelRef.current) observer.observe(sentinelRef.current);
    return () => observer.disconnect();
  }, [loadMore]);
```

```
    return (  
      <div>  
        {posts.map((post) => <article key={post.id}>{post.title}</article>  
        <div ref={sentinelRef} />  
        {loading && <p>Loading more...</p>}  
      </div>  
    );  
  }  
  
  // app/feed/actions.ts  
  'use server';  
  import { getPosts } from '@lib/posts';  
  
  export async function fetchMorePosts(cursor: string) {  
    return getPosts({ cursor, limit: 10 });  
  }  
}
```

Q117.

Implement a dynamic sitemap that pulls URLs from a data source.

MID

TYPESCRIPT

```
// app/sitemap.ts
import type { MetadataRoute } from 'next';
import { getPosts } from '@lib/posts';

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const posts = await getPosts();

  return [
    { url: 'https://site.com', lastModified: new Date() },
    { url: 'https://site.com/about', lastModified: new Date() },
    ...posts.map((post) => ({
      url: `https://site.com/blog/${post.slug}`,
      lastModified: post.updatedAt,
    })),
  ];
}
```

Q118.

Implement dark mode with SSR and no flash of incorrect theme.

MID

TSX

```
// app/layout.tsx
import { ThemeProvider } from 'next-themes';

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" suppressHydrationWarning>
      <body>
        <ThemeProvider attribute="class" defaultTheme="system" enableSystem>
          {children}
        </ThemeProvider>
      </body>
    </html>
  );
}

// components/ThemeToggle.tsx
'use client';
import { useTheme } from 'next-themes';

export default function ThemeToggle() {
  const { theme, setTheme } = useTheme();
  return (
    <button onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>
      Toggle theme
    </button>
  );
}
```

Q119. Implement a custom 404 page that also logs the missing path.

MID

TSX

```
// app/not-found.tsx
import { headers } from 'next/headers';
import { logMissingPath } from '@lib/logging';

export default async function NotFound() {
  const headersList = await headers();
  await logMissingPath(headersList.get('referer') ?? 'unknown');

  return (
    <div>
      <h1>404 - Page Not Found</h1>
      <p>Sorry, we couldn't find that page.</p>
    </div>
  );
}
```

Q120.

Implement authentication middleware that redirects unauthenticated users, preserving their intended destination.

SENIOR**TYPESCRIPT**

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { verifySessionToken } from '@lib/auth-edge';

const PROTECTED_PREFIXES = ['/dashboard', '/settings'];

export async function middleware(request: NextRequest) {
  const isProtected = PROTECTED_PREFIXES.some((p) =>
    request.nextUrl.pathname.startsWith(p)
  );
  if (!isProtected) return NextResponse.next();

  const token = request.cookies.get('session')?.value;
  const valid = token && (await verifySessionToken(token));

  if (!valid) {
    const loginUrl = new URL('/login', request.url);
    loginUrl.searchParams.set('redirectTo', request.nextUrl.pathname);
    return NextResponse.redirect(loginUrl);
  }

  return NextResponse.next();
}

export const config = { matcher: ['/dashboard/:path*', '/settings/:path*'] }
```

Q121.

Implement optimistic UI updates for a Server Action using useOptimistic.

SENIOR**TSX**

```
'use client';
import { useOptimistic } from 'react';
import { toggleLike } from './actions';

export default function LikeButton({ postId, initialLiked, initialCount }) {
  const [optimisticState, setOptimisticState] = useOptimistic(
    { liked: initialLiked, count: initialCount },
    (state, liked: boolean) => ({
      liked,
      count: state.count + (liked ? 1 : -1),
    })
  );

  async function handleClick() {
    setOptimisticState(!optimisticState.liked);
    await toggleLike(postId, !optimisticState.liked);
  }

  return (
    <button onClick={handleClick}>
      {optimisticState.liked ? 'Liked' : 'Like'} ({optimisticState.count})
    </button>
  );
}
```

Q122.

Implement a loading skeleton with Suspense for a slow data-fetching section.

MID

TSX

```
import { Suspense } from 'react';

async function RevenueChart() {
  const data = await getRevenueData(); // slow query
  return <Chart data={data} />;
}

function ChartSkeleton() {
  return <div className="skeleton" style={{ height: 300, borderRadius:
}

export default function DashboardPage() {
  return (
    <div>
      <h1>Dashboard</h1>
      <Suspense fallback={<ChartSkeleton />}>
        <RevenueChart />
      </Suspense>
    </div>
  );
}
```

Q123.

You're handed a slow page caused by a Server Component data-fetching waterfall — walk through fixing it.

SENIOR

Approach: first identify the waterfall — usually a chain of sequential `await` s where later calls don't actually depend on earlier results (check with the Next.js dev overlay or a request timeline in DevTools). Then:

TSX

```
// Before: 3 sequential round-trips
async function Page() {
  const user = await getUser();
  const orders = await getOrders(user.id);
  const recommendations = await getRecommendations(user.id);
  return <Dashboard user={user} orders={orders} recs={recommendations}>
}

// After: fetch user first (needed by the others), then run the
// two dependent calls in parallel instead of sequentially
async function PageFixed() {
  const user = await getUser();
  const [orders, recommendations] = await Promise.all([
    getOrders(user.id),
    getRecommendations(user.id),
  ]);
  return <Dashboard user={user} orders={orders} recs={recommendations}>
}
```

If the calls have a genuine dependency (need the first result to make the second call), the waterfall is unavoidable there — instead wrap the dependent, slower part in its own `<Suspense>` boundary so it doesn't block the rest of the page from streaming in first.

Q124.

Implement ISR for a blog with on-demand revalidation triggered when a post is published.

MID

TSX

```
// app/blog/[slug]/page.tsx
export const revalidate = 3600; // fallback: refresh at most hourly

export default async function BlogPost({ params }: { params: Promise<{
  const { slug } = await params;
  const post = await fetch(`https://cms.example.com/posts/${slug}`, {
    next: { tags: ['post-${slug}'] },
  }).then((r) => r.json());
  return <article>{post.content}</article>;
}

// app/api/revalidate/route.ts - called by the CMS webhook on publish
import { revalidateTag } from 'next/cache';

export async function POST(request: Request) {
  const { slug, secret } = await request.json();
  if (secret !== process.env.REVALIDATE_SECRET) {
    return new Response('Unauthorized', { status: 401 });
  }
  revalidateTag(`post-${slug}`);
  return Response.json({ revalidated: true });
}
```

Q125.**Implement a multi-step form using Server Actions that still works with JavaScript disabled (progressive enhancement).****SENIOR**

Because Server Actions used with `<form action={...}>` work as a real HTML form submission under the hood, the form remains functional even without client JS — Next.js just enhances it with client-side navigation/state when JS is available. Store step progress server-side (or in a hidden field / cookie) rather than only in client React state, so a full-page submission between steps still works correctly.

TSX

```
// app/signup/actions.ts
'use server';
import { redirect } from 'next/navigation';
import { cookies } from 'next/headers';

export async function submitStepOne(formData: FormData) {
  const email = formData.get('email') as string;
  (await cookies()).set('signup_email', email);
  redirect('/signup/step-2');
}

export async function submitStepTwo(formData: FormData) {
  const email = (await cookies()).get('signup_email')?.value;
  const password = formData.get('password') as string;
  await createAccount({ email, password });
  redirect('/welcome');
}

// app/signup/step-2/page.tsx
import { submitStepTwo } from '../actions';

export default function StepTwo() {
  return (
    <form action={submitStepTwo}>
      <input type="password" name="password" required />
      <button type="submit">Create account</button>
    </form>
  );
}
```